Cronograma de Entregas

Sistema Multiagente Inteligente de Cobrança com LLM, RAG e Machine Learning

Gabriel Forster Rocha

Jul. a dez. de 2026 — Engenharia de Software, Católica de Santa Catarina

1. Cronograma de entregas

Cada encontro tem **uma entrega pequena, verificável e acumulativa** — código rodando, dado gerado, ou seção de texto escrita. A coluna *Artefato* é o que se mostra no encontro; a coluna *Texto* é o que avança no artigo, para que a redação não fique concentrada no fim.

Situação em 07/11/2026. As entregas 1–14 estão integradas. As entregas 15, 16 e 17 estão construídas e medidas, em revisão nos *pull requests* #32–#37, e duas delas mudaram de forma (§2.4 e §2.5). O artigo da entrega 18 está redigido sobre esses resultados e aguarda a revisão. A entrega 19 — *tag*, *slides* e ensaio — é a que resta.

Bloco I — Fundação: requisitos e dados (30/07 a 27/08)

#	Data	Entrega	Artefato mostrado	Texto
1	**30/07**	**Definições.** Escopo fechado, este cronograma, repositório organizado (`article/`, `docs/`, RFC compilando)	RFC em PDF + cronograma aprovado	Proposta de portfólio revisada
2	06/08	**Requisitos e domínio.** Fluxo atual de cobrança mapeado, 10 RF + 6 RNF consolidados, restrições legais (CDC, LGPD, PROCON: horário, frequência, consentimento de gravação) e baselines de comparação definidos	Documento de requisitos + tabela de restrições legais	Seção de metodologia (etapa i)
3	13/08	**Dados brutos.** Fontes identificadas (cadastro, faturas, histórico de atraso, acordos, resultado de cobranças), extração do ERP, anonimização LGPD, dicionário de dados. Meta: ≥ 5.000 documentos históricos	Dataset anonimizado + dicionário de dados	Descrição da base de dados
4	20/08	**EDA e limpeza.** Estatísticas descritivas, distribuição do alvo, desbalanceamento medido, missings/outliers tratados, gráficos (atraso, inadimplência por segmento, sazonalidade, heatmap de correlação)	Notebook de EDA + 4 gráficos	Análise exploratória

#	Data	Entrega	Artefato mostrado	Texto
5	27/08	**Features e splits.** *Feature engineering* (dias médios de atraso, taxa histórica, frequência recente, valor relativo, tendência de pagamento), encoding, escala, split 70/15/15 estratificado **respeitando ordem temporal** (sem vazamento)	Pipeline de features versionado + 3 conjuntos	Justificativa das features

Bloco II — Módulo preditivo (03/09 a 17/09)

#	Data	Entrega	Artefato mostrado	Texto
6	03/09	**Baseline.** Regressão logística treinada na Tarefa 1 (propensão ao pagamento), métricas completas (acurácia, precisão, revocação, F1, AUC-ROC) + matriz de confusão	Tabela de métricas do baseline	Baseline preditivo
7	10/09	**Modelos fortes, Tarefa 1.** Random Forest e XGBoost/LightGBM com validação cruzada (k=5), *tuning* de hiperparâmetros e tratamento de desbalanceamento (SMOTE vs. *class weights*); modelo campeão escolhido	Comparativo de algoritmos + curvas ROC/PR	Resultados da Tarefa 1
8	17/09	**Tarefa 2 + API.** Modelo de previsão de inadimplência futura (documentos não vencidos) treinado e avaliado, interpretabilidade (*feature importance* + SHAP), modelo serializado e exposto por **API de inferência** que devolve o *ranking* de prioridade	`POST /score` respondendo + gráfico SHAP	Resultados da Tarefa 2 e interpretabilidade

Bloco III — Arquitetura e módulo responsivo (24/09 a 15/10)

#	Data	Entrega	Artefato mostrado	Texto
9	24/09	**Arquitetura multiagente.** Quatro camadas definidas, papéis dos três agentes, contrato de eventos, fila de mensagens e regra de prioridade (responsivo > proativo); esqueleto do **orquestrador central** rodando com agentes *stub*	Diagrama + orquestrador roteando evento de ponta a ponta	Seção de arquitetura
10	01/10	**RAG.** Ingestão (PDF/DOCX/CSV/TXT) de políticas, *scripts* de negociação, tabelas de parcelamento e FAQ; *chunking* (500–1000 tokens, *overlap* 50–100), *embeddings*, indexação vetorial e *retrieval* top-k avaliado por precisão/revocação	Consulta ao índice retornando trechos corretos	Pipeline RAG
11	08/10	**Agente responsivo (texto).** *System prompt*, classificação de intenção (consulta, 2ª via, negociação, contestação, confirmação de pagamento), resposta ancorada no RAG, critérios de escalação para humano	Conversa completa por texto, latência < 5 s medida	Módulo responsivo

#	Data	Entrega	Artefato mostrado	Texto
12	15/10	**Canal + áudio.** Canal de chat simulado no lugar da WhatsApp Business API — a integração exigia conta comercial, aprovação de *templates* e cobrança, e não acrescentaria nenhuma decisão nova aos agentes (ver §2); transcrição de áudio (OGG/OPUS → STT) entregue pelo PR #17, com tratamento de erro (áudio inaudível, curto, *timeout*)	Áudio respondido corretamente no canal de chat	Integração de voz (offline)

Bloco IV — Módulo ativo e telefonia (22/10 a 05/11)

#	Data	Entrega	Artefato mostrado	Texto
13	22/10	**Detecção + regras.** *Job* periódico de documentos vencidos, consumo do *ranking* preditivo para cobrança preventiva, motor de regras (horário permitido, frequência máxima, canal, *cooldown*, limite de tentativas)	Fila de cobrança populada com prioridade preditiva	Módulo ativo (parte 1)
14	29/10	**Abordagem multicanal por texto.** *Templates* por estágio de atraso (1–7, 8–30, 31–60, 60+ dias) personalizados por LLM, envio pelo **canal de chat simulado**, *tracking* de status (enviada/entregue/lida/respondida, com `entregue`/`lida` informados pelo cliente e não pela operadora), respeito imediato a *opt-out*, transferência ao agente responsivo quando o cliente responde	Escalonamento progressivo executando em ambiente de teste	Módulo ativo (parte 2)
15	05/11	**Chamada por voz em tempo real (WebRTC/LiveKit).** No lugar da telefonia comutada: fluxo TTS de abertura → STT em tempo real → resposta → encerramento, com *barge-in*, aviso de gravação que não pode ser pulado e registro completo da chamada. O mesmo agente é levado a um número real por SIP, sem alteração no agente (ver §2). **Entregue reduzida a prova de conceito:** sem *barge-in* e sem SIP (ver §2.4)	Chamada em tempo real demonstrada no navegador + tabela de tempo de turno	Integração de voz (tempo real)

Bloco V — Avaliação e fechamento (12/11 a 03/12)

#	Data	Entrega	Artefato mostrado	Texto
16	12/11	**Testes e início do experimento.** Testes unitários (parser, *features*, motor de regras, intenção, *templates*) com ≥ 80% das funções críticas cobertas, testes de integração dos três fluxos e dos serviços externos; **experimento controlado iniciado** contra os dois baselines (manual e semiautomatizado sem ML). **Entregue:** as 150 funções críticas executadas pela suíte; o experimento é uma simulação, pela decisão D1 (ver `docs/experiment-protocol.md`)	Suíte de testes verde + experimento coletando dados	Protocolo experimental
17	19/11	**Resultados consolidados.** Métricas preditivas finais vs. baseline, qualidade do RAG (*retrieval* + fidelidade/completude/alucinação das respostas), indicadores operacionais (taxa de recuperação, tempo médio de resolução, custo por contato, ≥ 100 interações concorrentes), análise estatística. **Entregue:** o braço LLM medido num modelo local (ver §2.5) e a concorrência medida em memória, com o respondedor por *templates*	Tabelas e gráficos finais de resultados	Seção de resultados e discussão
18	**26/11**	**Revisão final.** Artigo completo revisado (resumo/abstract, referências, tabelas, figuras), limitações e trabalhos futuros, README de reprodução; ajuste dos pontos de revisão apontados	Artigo em PDF, versão candidata	Texto integral fechado
19	**03/12**	**Trabalho finalizado.** Versão final entregue, repositório com *tag* de entrega, apresentação/defesa preparada e ensaiada	PDF final + *slides* + demonstração	—

---

2. Mudanças de escopo

Cinco entregas deste cronograma mudaram de forma depois de aprovadas. Elas estão reunidas aqui, com data, o que foi substituído, por quê e o que deixa de existir nos resultados — para que o leitor encontre as substituições em um lugar só, e não as descubra espalhadas pelo texto.

A decisão foi registrada em **22/10/2026**, antes de a implementação ser integrada, e o argumento completo está em `docs/channels.md`.

O critério. A pergunta de pesquisa é se uma arquitetura multiagente com LLM, RAG e um modelo preditivo consegue conduzir a cobrança de forma autônoma, segura e dentro das restrições legais. Isso é respondido pelo que os **agentes decidem**, não por qual API comercial transporta os bytes. Uma integração só entra no escopo se acrescentar uma decisão nova a algum agente; se acrescenta apenas transporte, fila de aprovação e contrato de faturamento, ela é substituível sem perda para a tese. As três primeiras mudanças abaixo seguem esse critério, decididas em 22/10. As duas últimas (§2.4 e §2.5) vieram depois, de outra natureza: não trocam um transporte por outro, reduzem o que foi entregue ou o que foi medido — e por isso dizem com o mesmo cuidado o que deixa de ser afirmado.

2.1 WhatsApp Business API → canal de chat simulado

Entrega 12 (15/10) e metade da entrega 14 (29/10).

O que foi substituído. A integração com a WhatsApp Business API, com *webhooks*, *templates* aprovados pela Meta e conta comercial, por um canal de chat embedável — o mesmo widget já existente, agora com balões, áudios no fio da conversa, status de entrega e conversas iniciadas pelo sistema.

Por quê. A integração exige conta comercial, aprovação de *templates* e relação de faturamento, e **não acrescenta nenhuma decisão nova a nenhum agente**. O risco técnico real está no *contrato de canal*. O que está provado hoje é preciso e mais estreito do que um segundo adaptador provaria: **nenhum agente consegue importar um canal** — há um teste que varre o pacote de agentes com um analisador sintático e falha se qualquer um deles importar um canal por qualquer declaração de import, e esse teste tem o seu próprio teste, que planta cada grafia e verifica que o alarme dispara. É dessa propriedade que um futuro adaptador de WhatsApp depende, e integrar a API não a estabeleceria: com um canal só não há nada com que ser inconsistente. Um segundo adaptador seria evidência mais forte, e é de onde ela virá (entrega 15).

O que falta nos resultados. Recibos de entrega fim a fim reais: `entregue` e `lida` são **informados pelo cliente** (o navegador avisa que renderizou e que exibiu a mensagem), não confirmados pela operadora. Um navegador pode mentir ou simplesmente nunca avisar, e nada é inferido por temporizador — um recibo simulado seria um número com cara de medido. Só `respondida` é observada pelo servidor. Também ficam de fora as restrições de redação impostas pela aprovação de *templates* da Meta.

2.2 Telefonia comutada → chamada por voz em tempo real (WebRTC/LiveKit)

Entrega 15 (05/11).

O que foi substituído. Número *outbound*, tronco SIP e operadora, por uma chamada de voz em tempo real no navegador, com detecção de fim de turno, interrupção pelo cliente (*barge-in*) e aviso de gravação. A chamada foi entregue menor do que esta descrição: ver §2.4.

Por quê. Os problemas interessantes de uma ligação automatizada são a alternância de turnos, a interrupção, o orçamento de latência e a abertura legal. Os quatro existem igualmente numa chamada WebRTC; nenhum deles é criado pelo SIP. O LiveKit documenta levar o mesmo agente a um número real por SIP sem alterar o agente, o que torna a substituição **reversível em projeto** — mas isso é uma intenção de arquitetura, **não validado** neste trabalho: nenhum tronco SIP foi contratado nem testado. Dizer onde a afirmação termina faz parte da defesa.

O que falta nos resultados. Perda de pacotes de rede real e *codecs* de áudio de operadora, e o que eles custam à acurácia do reconhecimento. O módulo de voz mede o que o erro de reconhecimento custa ao restante do sistema (`docs/voice.md`), o que dá uma forma honesta de discutir áudio degradado de telefonia — mas é uma varredura sobre um modelo de erro, não uma medição de uma linha telefônica.

2.3 Entrega proativa por *push* → conversa semeada

Entrega 14 (29/10).

O que foi substituído. O envio ativo de uma mensagem que chega ao cliente, por uma mensagem que fica **esperando**: o agente proativo deposita o contato na caixa do cliente e ele aparece no topo do fio quando o widget é aberto.

Por quê. Não há transporte de *push* sem a conta de um provedor — que é exatamente o que 2.1 removeu. A alternativa seria afirmar uma entrega que o sistema não realiza. A composição, a checagem de conformidade, a supressão por *opt-out* e o rastreamento de status são os

pedaços difíceis e estão todos construídos. O comando `make contacts` executa o caminho inteiro — agente proativo, regras, composição, conformidade e caixa de entrada — e relata quantos contatos foram semeados, suprimidos ou recusados por serem de outro canal; a demonstração ao vivo é `make serve ARGS="--seed-demo"`, em que o próprio servidor semeia o canal que ele atende, porque a caixa de entrada vive em memória e não atravessa processos. O transporte de chegada ao vivo (SSE ou *polling*, contra o mesmo contrato) fica registrado como trabalho futuro.

O que falta nos resultados. Qualquer métrica de alcance de campanha proativa: um fio semeado não é uma mensagem que chegou, e um cliente que nunca abre o widget nunca recebe nada. O *opt-out* é verificado duas vezes por causa disso — no agendamento e de novo no momento da entrega, porque o cliente pode se descadastrar entre um e outro.

2.4 Chamada por voz completa → prova de conceito reduzida

Entrega 15 (05/11), concluída depois da data.

O que foi substituído. A chamada descrita em 2.2, com interrupção pelo cliente e um caminho até um número real por SIP, por uma prova de conceito: um LiveKit hospedado localmente, em modo de desenvolvimento e sem conta de fornecedor; o aviso de gravação tocado antes de tudo, com a fala do cliente durante ele descartada; Whisper local para reconhecer; o **mesmo** agente responsivo do chat; Piper local para sintetizar (`pt_BR-faber-medium`); e o registro completo da chamada, com o áudio apagável e a transcrição mantida. O fim do turno é detectado por energia, não por um modelo treinado.

Por quê. É exatamente o que §4 previa: a chamada por voz era a primeira candidata a virar prova de conceito, e virou. O que a tese precisa da voz — a alternância de turnos, o aviso de gravação obrigatório, o orçamento de latência e a prova de que um segundo canal reaproveita o agente sem alterá-lo — está construído e medido. A síntese é local pelo mesmo motivo do Whisper: a resposta lê em voz alta a dívida de uma pessoa identificada, e um serviço de TTS na nuvem passaria a tratar esse dado. A integração usa o SDK de baixo nível do LiveKit, e não o LiveKit Agents, porque o *framework* reconheceria a fala por conta própria e contornaria a escalação por baixa confiança e a confirmação de valores falados.

O que falta nos resultados. A tabela de tempo de turno existe (`docs/voice-call.md`): mediana de **2.184 ms** medida sobre WebRTC, do fim da fala ao primeiro áudio da resposta, em CPU, com Whisper `small` e o respondedor por *templates*. Ficam de fora a interrupção pelo cliente, o SIP e a telefonia comutada, a discagem ativa e uma medição da divergência entre canais, que é zero por construção mas não foi medida. Uma pessoa falando ao microfone do navegador **não foi verificada** — só fala sintetizada atravessando uma sala real. E a medição mostrou um limite que a arquitetura não resolve sozinha: lida em voz alta, a resposta por *templates* dura uma mediana de **102 s**, porque cita trechos inteiros. A chamada só é usável com um respondedor generativo atrás dela, e o canal não pode encurtar o que o agente diz.

2.5 Braço LLM hospedado → modelo local

Entregas 11 e 17.

O que foi substituído. A geração por um modelo hospedado (Claude) nas três comparações que têm um braço LLM — o respondedor, a reescrita de consultas em conversas de vários turnos e a composição da mensagem ativa — por um modelo pequeno rodando localmente: `qwen2.5:3b`, servido pelo Ollama no `docker-compose`, em CPU, por um cliente que fala o protocolo da OpenAI.

Por quê. Não havia chave de API de um provedor hospedado, e os braços LLM nunca tinham rodado. O cliente novo atende qualquer servidor compatível (Ollama, vLLM, OpenRouter, Groq), então a troca é de configuração: quando houver uma chave, o mesmo comando

acrescenta uma linha às tabelas, sem código novo. Cada relatório gerado registra o provedor, o modelo e a data do braço LLM.

O que falta nos resultados. Um número de modelo hospedado. Os resultados publicados são um **piso**, não uma estimativa do que um modelo grande faria: no respondedor, fidelidade de 0,69 contra 1,0 do *template* e p95 de 11,1 s contra um orçamento de 5 s; na mensagem ativa, 55% das mensagens conformes contra 100% do *template*, embora a distinção entre mensagens mais que dobre (0,307 → 0,672); na reescrita, empate com a concatenação simples (0,800), porque o modelo devolveu quase todas as perguntas de seguimento sem reescrevê-las. A conclusão que o artigo tira disso é restrita ao modelo medido.

2.6 O que não mudou

A anotação é tão importante quanto as substituições. O vocabulário de dados do ERP (`whatsapp`, `phone`, `email`, `sms` em `preferred_channel` e no histórico de contatos) **não foi renomeado**: aquilo é um fato sobre o cliente e sobre contatos que de fato aconteceram, e reescrevê-lo para combinar com o sistema corromperia a base com que o modelo preditivo é treinado. Os canais de entrega do sistema (`chat`, `voice`, `sms`, `email`) são um vocabulário separado, por decisão.

---

3. Relação com o cronograma do RFC

O RFC (Tabela 3) previa 24 semanas em 8 fases; este cronograma distribui as mesmas 8 fases nos 19 encontros, sobrepondo blocos que podem correr em paralelo (arquitetura durante o fim do módulo preditivo; voz sobre o módulo ativo) e reservando as três últimas semanas para avaliação, redação e defesa.

Fase do RFC	Encontros
1 — Requisitos e análise do domínio	30/07, 06/08
2 — Coleta e preparação de dados	13/08, 20/08, 27/08
3 — Módulo preditivo de ML	03/09, 10/09, 17/09
4 — Arquitetura multiagente	24/09
5 — Módulo responsivo (RAG + LLM)	01/10, 08/10, 15/10
6 — Módulo ativo/proativo	22/10, 29/10
7 — Integração de voz	15/10 (STT offline), 05/11 (voz em tempo real, WebRTC)
8 — Testes, experimentos e avaliação	12/11, 19/11
Redação final e defesa	26/11, 03/12

---

4. Riscos e plano de contingência

Risco	Impacto	Contingência
Volume ou qualidade insuficiente dos dados históricos	Bloqueia as entregas 6–8	Antecipar a extração (13/08) e, se necessário, usar base pública de *credit scoring* para validar o pipeline, mantendo a base real apenas para o modelo final
Aprovação de *templates* na Meta / conta de telefonia	Atrasava 29/10 e 05/11	**Risco materializado.** Resolvido pelas substituições de §2: canal de chat simulado e chamada por voz em tempo real, ambas atrás do mesmo contrato de canal; a chamada foi entregue como prova de conceito reduzida (§2.4)
Latência das chamadas com STT em tempo real	Compromete a entrega 15	Whisper roda localmente por decisão de LGPD, então a latência é medida e publicada (tabela de tempo de turno) em vez de escondida. **Medida:** mediana de 2.184 ms por turno; o gargalo que apareceu foi outro — a duração da resposta falada (§2.4)
Experimento operacional com amostra pequena	Enfraquece a entrega 17	Relatar tamanho de efeito e intervalo de confiança; complementar com métricas técnicas e comparação de custo por contato. **Materializado de forma mais forte:** sem dados operacionais (decisão D1), o experimento é uma simulação com 30 replicações pareadas e intervalos de 95%, e o artigo diz isso
Custo de APIs de LLM, STT/TTS e telefonia	Afeta 08/10 em diante	**Resolvido localmente:** Whisper, Piper e o modelo de linguagem rodam na máquina, sem custo por chamada; o preço é a qualidade e a latência de um modelo pequeno (§2.5)

Escopo mínimo garantido (se algo precisar cair): os módulos responsivo e preditivo mais o fluxo proativo por texto são obrigatórios; a chamada por voz em tempo real é a primeira candidata a virar prova de conceito reduzida, com a limitação documentada no artigo. Foi o que aconteceu (§2.4), e os três módulos obrigatórios foram entregues inteiros.